

FLOWDAR

Specifications Backend — v2

Angular Talent Lab 2026 — Orange Digital Center Douala

1. Stack technique recommandee

Composant	Technologie	Justification
Runtime	Node.js 20+ LTS	Performances async pour les cron jobs et requetes GPS en parallele
Framework API	Express.js	Simple, bien documente, parfait pour les endpoints REST
Base de donnees	PostgreSQL 16+ + PostGIS	Requetes geospaciales (altitude, proximite eau, polygones de quartiers)
Scraping	Puppeteer + Cheerio	Puppeteer pour sites JS dynamiques (ONACC), Cheerio pour HTML statique
IA	Gemini API	Analyse signalements texte + resumes intelligents (fonctionnalite Could)
Taches planifiees	node-cron	Jobs automatiques toutes les 15-30 min
Requetes paralleles	Promise.all	Envoyer toutes les requetes meteo GPS simultanement sans bloquer
Deploiement	Railway	Gratuit pour projets de formation, PostgreSQL integre

2. Modele de donnees PostgreSQL

Mise a jour v2 : ZoneARisque enrichie (altitude, proximite eau, seuil_score, nb_occurrences). Table HistoriqueMeteoAlertes ajoutee pour l'apprentissage des seuils.

Table : zones_a_risque

Champ	Type	Description
id	UUID PK	Identifiant unique
nom_quartier	VARCHAR(100)	Ex: Ndokotti, Bepanda, Makepe, Logpom, PK8-PK14, Ndog-Bong, Melen
lat	DECIMAL(10,7)	Latitude GPS precise du centroide du quartier
lng	DECIMAL(10,7)	Longitude GPS precise du centroide du quartier
polygone	GEOMETRY(POLYGON)	Contour geospatial PostGIS du quartier

Champ	Type	Description
altitude_m	INTEGER	Altitude en metres (Google Elevation API) — zone basse = risque eleve
proximite_eau_m	INTEGER	Distance en metres du cours d'eau le plus proche (OpenStreetMap)
niveau_risque_base	ENUM	faible / moyen / eleve — issu ONACC ou signalements citoyens
seuil_score	INTEGER	Score minimum (0-100) pour declencher une alerte dans ce quartier specifique
nb_occurrences	INTEGER DEFAULT 0	Nombre d'inondations enregistrees depuis la creation
source	ENUM	onacc / historique_terrain / signalement_citoyen
date_creation	TIMESTAMP	Date d'ajout de la zone

Table : alertes

Champ	Type	Description
id	UUID PK	Identifiant unique
type	ENUM	preventive (risque prevu) / active (inondation en cours)
source	ENUM	auto / citoyen / onacc
zone_id	UUID FK	Lien vers zones_a_risque
score	INTEGER 0-100	Score calcule au moment de la creation
niveau	ENUM	leger (30-60) / moyen (60-85) / dangereux (>85) — derive du score
heure_debut	TIMESTAMP	Heure de creation de l'alerte
heure_prevue	TIMESTAMP NULL	Heure prevue (alertes preventives uniquement)
nb_confirmations	INTEGER DEFAULT 0	Confirmations citoyennes recues
statut	ENUM	actif / resolu / en_resolution
photo_url	VARCHAR NULL	URL Firebase Storage
firestore_id	VARCHAR	ID Firestore pour synchronisation temps reel

Table : historique_meteo_alertes (NOUVELLE)

Champ	Type	Description
id	UUID PK	Identifiant unique
zone_id	UUID FK	Quartier concerne
date	TIMESTAMP	Date et heure de la mesure
pluie_mm_h	DECIMAL	Precipitations en mm/h pour ce quartier (requete GPS)

Champ	Type	Description
humidite_pct	INTEGER	Humidite du sol en % (OpenWeatherMap)
pluie_cumul_3h	DECIMAL	Pluie cumulee sur les 3 dernieres heures
score_calcule	INTEGER	Score d'inondation calcule a ce moment
alerte_generee	BOOLEAN	Une alerte a-t-elle ete generee ?

Table : signalements_bruts

Champ	Type	Description
id	UUID PK	Identifiant unique
texte_brut	TEXT	Contenu brut saisi par le citoyen ou scrape
lat_signal	DECIMAL NULL	Coordonnees GPS du signalement (si zone inconnue)
lng_signal	DECIMAL NULL	Coordonnees GPS du signalement (si zone inconnue)
source	VARCHAR	citoyen / twitter / facebook / onacc
analyse_gemini	JSONB NULL	Resultat JSON de l'analyse Gemini
alerte_generee_id	UUID NULL FK	Alerte generee si signalement valide
created_at	TIMESTAMP	Date de collecte

Table : confirmations

Champ	Type	Description
id	UUID PK	Identifiant unique
alerte_id	UUID FK	Alerte confirmee ou resolue
user_uid	VARCHAR	UID Firebase Auth du citoyen
type	ENUM	confirme / resolu
created_at	TIMESTAMP	Heure de la confirmation

Table : utilisateurs

Champ	Type	Description
uid	VARCHAR PK	UID Firebase Auth
nom	VARCHAR	Nom de l'utilisateur
email	VARCHAR	Email de connexion
quartier_domicile	VARCHAR	Quartier de residence

3. Logique de calcul du score d'inondation

C'est le coeur du backend. Chaque quartier recoit son propre score 0-100 base sur 4 facteurs combines. Le seuil de declenchement est different pour chaque quartier.

Requetes meteo GPS par quartier

Le backend ne fait PAS une seule requete pour 'Douala'. Pour chaque zone, il fait une requete distincte avec les coordonnees GPS precises :

```
// Pour chaque zone dans zones_a_risque, requete individuelle :  
GET /weather?lat=4.0511&lon=9.7679 → Ndokotti  
GET /weather?lat=4.0622&lon=9.7510 → Bepanda  
GET /weather?lat=4.0833&lon=9.7500 → Makepe  
// ...  
  
// Toutes les requetes envoyees en parallele :  
const resultats = await Promise.all(  
  zones.map(zone => fetchMeteoGPS(zone.lat, zone.lng))  
);
```

Calcul du score (0-100) par quartier

Facteur	Donnee source	Poids	Calcul
Meteo temps reel	OpenWeatherMap /weather GPS	40%	pluie_mm_h / seuil_zone * 40 + bonus humidite + bonus cumul 3h
Historique	Table historique_meteo_alertes	30%	Ratio alertes passees dans conditions similaires * 30
Signalements citoyens	Table signalements_bruts actifs < 1h	20%	min(nb_signalements / 3, 1) * 20
Geographie	altitude_m + proximite_eau_m	10%	Zone basse + pres d'un cours d'eau = score geo eleve

Score calcule	Niveau	Decision
< 30	Aucun	Pas d'alerte
30 a 60	Leger	Alerte preventive
60 a 85	Moyen	Alerte active
> 85	Dangereux	Alerte critique

Decouverte dynamique de nouvelles zones

Quand des signalements citoyens arrivent sur une zone non repertoriee :

1. Citoyen signale une zone hors de la liste connue (avec coordonnees GPS)
2. Backend verifie : y a-t-il 5+ signalements sur ces coordonnees en < 1h ?
OUI → Creer automatiquement une entree dans zones_a_risque
source = 'signalement_citoyen'
niveau_risque_base = 'inconnu'
seuil_score = 50 (par defaut jusqu'a apprentissage)
3. Apres 3 occurrences d'inondation sur cette zone :
→ Mettre a jour niveau_risque_base et seuil_score
selon les donnees de historique_meteo_alertes

4. Endpoints API

Alertes

Methode	Endpoint	Description	Reponse
GET	/api/alertes	Toutes les alertes actives	Array avec coordonnees GPS, score, niveau
GET	/api/alertes/:id	Detail d'une alerte	Objet alerte + confirmations
POST	/api/alertes/signaler	Signalement citoyen	Alerte creee apres analyse Gemini
POST	/api/alertes/:id/confirmer	Confirmer une alerte	nb_confirmations + 1
POST	/api/alertes/:id/resoudre	Marquer resolue	Statut -> resolu
GET	/api/alertes/historique/:quartier	Historique par quartier	Array alertes passees

Zones a risque

Methode	Endpoint	Description	Reponse
GET	/api/zones-a-risque	Toutes les zones	Array avec coordonnees GPS, altitude, seuil_score
GET	/api/zones-a-risque/:id	Detail d'une zone	Zone + historique alertes + score actuel

Score et meteo

Methode	Endpoint	Description	Reponse
GET	/api/scores	Score actuel de toutes les zones	Array zone_id + score + niveau
GET	/api/meteo/actuelle	Meteo actuelle par zone	Array zone_id +

Methode	Endpoint	Description	Reponse
			pluie_mm_h + humidite
GET	/api/meteo/previsions	Previsions 6h par zone	Array zone_id + heure_prevue + score_previsionnel

Itineraire

Methode	Endpoint	Description	Reponse
POST	/api/itineraire	Zones actives a eviter	Array de coordonnees GPS des zones alertees (passe a Google Maps cote Angular)

5. Cron Jobs automatiques

Job	Frequence	Ce qu'il fait	Sortie
score-update	Toutes les 30 min	Requetes meteo GPS pour chaque zone en parallele (Promise.all). Calcule le score combine. Cree ou met a jour les alertes. Stocke dans historique_meteo_alertes	Alertes ecrites dans PostgreSQL + Firestore
zone-discovery	Toutes les 30 min	Verifie les signalements citoyens hors zones connues. Si 5+ signalements sur meme GPS en < 1h : cree nouvelle zone	Nouvelle entree zones_a_risque si necessaire
seuil-update	Toutes les 24h	Analyse historique_meteo_alertes. Ajuste seuil_score de chaque zone selon les occurrences reelles	Seuils mis a jour dans zones_a_risque
scraper-onacc	Toutes les heures	Puppeteer scrape onacc.cm. Extrait bulletins d'alerte officiels	Signalements bruts stockes
auto-resolve	Toutes les heures	Alertes sans confirmation depuis > 3h : statut -> resolu	Alertes obsoletes retirees de la carte
sync-firestore	A chaque modification PostgreSQL	Synchronise les alertes actives + scores vers Firestore	Angular notifie via onSnapshot

6. Synchronisation PostgreSQL vers Firestore

PostgreSQL est la source de verite. Firestore est le canal de diffusion temps reel vers Angular.

```

Backend calcule un nouveau score et cree une alerte dans PostgreSQL
|
v
Backend ecrit la meme alerte dans Firestore (collection 'alertes')
Champs essentiels : id, zone_id, score, niveau, statut, lat, lng
|
v
Angular ecoute via onSnapshot :
firestore.collection('alertes').where('statut','=','actif')
|
v
Nouveau marqueur sur Google Maps sans rechargement de page

```

7. Variables d'environnement

Variable	Description	Obtention
OPENWEATHER_API_KEY	Cle API OpenWeatherMap	openweathermap.org — compte gratuit
GOOGLE_ELEVATION_API_KEY	Cle API Google Elevation	console.cloud.google.com
GEMINI_API_KEY	Cle API Gemini	aistudio.google.com — compte Google gratuit
DATABASE_URL	URL PostgreSQL	Railway lors du deploiement
FIREBASE_SERVICE_ACCOUNT	JSON credentials Firebase Admin SDK	Console Firebase -> Parametres -> Comptes de service
FRONTEND_URL	URL Angular (pour CORS)	Firebase Hosting URL

8. Checklist de livraison backend

1. API REST operationnelle avec tous les endpoints de la section 4
2. Cron jobs actifs : score-update, zone-discovery, seuil-update, scraper-onacc, auto-resolve, sync-firestore
3. Calcul du score combine 4 facteurs implemente pour chaque quartier
4. Requetes meteo GPS individuelles par quartier (Promise.all)
5. Decouverte dynamique de nouvelles zones (5+ signalements -> creation auto)
6. Mise a jour automatique des seuils via historique_meteo_alertes
7. Synchronisation PostgreSQL -> Firestore fonctionnelle
8. Zones precharges : Ndokotti, Bepanda, Makepe, Logpom, PK8-PK14, Ndog-Bong, Melen
9. Variables d'environnement documentees dans .env.example
10. API deployee et accessible via URL publique pour Angular

